



UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Asignatura: Análisis y Diseño de Software – NRC: 30723

Tema: Patrones Comportamentales ITERATOR y MEDIATOR

Grupo Número: 7

Integrantes:

Diana Guerra

Paul Moreno

Leonel Tipan

Fecha: 21 de mayo del 2026

Sangolquí – Ecuador

Patrón Iterator

Nombre del patrón: Iterator

Descripción: El patrón Iterator que permite recorrer los elementos de una colección sin exponer su representación subyacente, ya sea que internamente funcione como una lista, pila, árbol u otra estructura de datos

Descripción del problema: En el caso de uso “Mostrar todo”, el Administrador necesita visualizar un listado de todos los estudiantes registrados en el sistema. El problema surge de cómo se almacena este conjunto de estudiantes. La colección podría estar guardada en una lista simple, pero si el sistema escala, podría estructurarse en un árbol complejo para ordenar a los estudiantes alfabéticamente o por edades. Si el sistema o el cliente intenta acceder a estos registros directamente, tendría que conocer los detalles de cómo están organizados, lo cual acoplaría el código a implementaciones específicas y limitaría la flexibilidad. Además, si añadimos toda la lógica de los distintos tipos de recorrido (por ejemplo, buscar por edad, buscar en orden alfabético) directamente a la colección de estudiantes, esta clase perdería su responsabilidad principal de almacenar eficientemente la información.

Ventajas:

- **Principio de responsabilidad única:** Permite limpiar el código de la clase ControlEstudiante y de la propia colección, separando las voluminosas lógicas de recorrido en clases independientes.
- **Principio de abierto/cerrado:** Se pueden implementar nuevos tipos de recorridos (por ejemplo, un IteradorEstudiantesEgresados o un IteradorPorEdad) sin tener que romper o modificar el código que ya existe para mostrar a los estudiantes.
- **Múltiples recorridos simultáneos:** Es posible recorrer la misma colección de estudiantes en paralelo, ya que cada objeto iterador mantiene de forma independiente su propio estado y posición de iteración.
- **Pausar recorridos:** Se puede retrasar una iteración sobre los registros de los estudiantes y continuarla en el punto exacto donde se dejó cuando sea necesario

Desventajas:

- **Complejidad innecesaria (Overkill):** Si es una aplicación pequeña y la lista de estudiantes se basa siempre en un arreglo o lista sencilla y predecible, aplicar este patrón puede resultar excesivo y añadir sobrecarga al diseño.
- **Pérdida de eficiencia:** Dependiendo de las colecciones especializadas que se utilicen para los estudiantes, usar un iterador abstracto podría llegar a ser menos eficiente que acceder y recorrer los elementos de forma directa.

Patron Mediator

Nombre del Patron: Mediator

Definición: El patrón Mediator que permite reducir las dependencias entre objetos. Este patrón restringe las comunicaciones directas entre los distintos componentes, forzándolos a colaborar e interactuar únicamente a través de un objeto central o mediador.

Descripción del problema: Se cuenta con una interfaz gráfica para gestionar la información. Esta pantalla se compone de varios controles visuales: botones para Agregar, Actualizar y Eliminar, una tabla para "Mostrar todo" y campos de texto para ingresar el ID, Nombre y Edad del estudiante. A medida que el formulario incorpora la lógica de los requisitos funcionales, las relaciones entre estos elementos se vuelven complejas y caóticas. Por ejemplo:

- Al seleccionar un estudiante en la tabla, los campos de texto deben rellenarse automáticamente con su ID, Nombre y Edad.
- Al hacer clic en el botón "Agregar estudiante", el sistema debe primero invocar la validación de los datos revisando los campos de texto. Si es exitoso, debe actualizar la tabla para mostrar el nuevo registro y luego limpiar los campos.

Ventajas

- **Principio de responsabilidad única:** Permite extraer las complejas comunicaciones entre los distintos elementos del formulario a un único lugar (el mediador), haciendo el código mucho más fácil de comprender y mantener.
- **Reutilización de componentes:** Al aplicar este patrón, los componentes individuales ya no conocen ni dependen de los otros controles de la interfaz, lo que permite utilizarlos en otras pantallas de la aplicación simplemente vinculándolos a un nuevo mediador.
- **Reducción de acoplamiento:** Minimiza significativamente las dependencias entre los componentes gráficos.

Desventajas

- **El anti-patrón del Objeto Todopoderoso ("God Object"):** Con el paso del tiempo y a medida que el CRUD crezca en funcionalidades o controles, el objeto MediatorCRUDEstudiantes corre el riesgo de acumular demasiada lógica, evolucionando hasta convertirse en un objeto todopoderoso y monolítico que sea complejo de gestionar

Modelado Mediator:

```
@startuml
Diagrama_Mediator_CRUD_Estudiante
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

interface "IMediadorCrudEstudiante" <<mediator>> {
    + agregar()
    + actualizar()
    + eliminar()
    + mostrarTodo()
    + estudianteSeleccionado(id: String)
}

abstract class "ComponenteCrudEstudiante" <<boundary>> {
    # mediador: IMediadorCrudEstudiante
    + setMediador(mediador: IMediadorCrudEstudiante)
}

class "FormularioDatosEstudiante" <<boundary>> {
    - txtId: String
    - txtNombre: String
    - txtEdad: String
    + obtenerId(): String
    + obtenerNombre(): String
    + obtenerEdad(): int
    + cargarEstudiante(estudiante: Estudiante)
    + limpiarCampos()
    + mostrarMensaje(mensaje: String)
}

class "TablaEstudiantes" <<boundary>> {
    + mostrarTabla(estudiantes: List<Estudiante>)
    + obtenerIdSeleccionado(): String
    + limpiarTabla()
}

class "PanelAccionesEstudiante" <<boundary>> {
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
    + habilitarEdicion()
    + deshabilitarEdicion()
}

class "MediadorCrudEstudiante" <<control>> {
    - formulario: FormularioDatosEstudiante
    - tabla: TablaEstudiantes
    - acciones: PanelAccionesEstudiante
    - control: ControlEstudiante

    + agregar()
    + actualizar()
    + eliminar()
    + mostrarTodo()
    + estudianteSeleccionado(id: String)
}

class "ControlEstudiante" <<control>> {
    + agregarEstudiante(id: String, nombre: String, edad: int): Resultado
}
```

```

+ actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
+ eliminarEstudiante(id: String): Resultado
+ mostrarTodos(): List<Estudiante>
+ validarDatos(id: String, nombre: String, edad: int): boolean
}

class "RepositorioEstudiante" <<repository>> {
+ existeId(id: String): boolean
+ guardar(estudiante: Estudiante): void
+ buscarPorId(id: String): Estudiante
+ actualizar(estudiante: Estudiante): void
+ eliminar(id: String): void
+ listarTodos(): List<Estudiante>
}

class "Estudiante" <<entity>> {
- id: String
- nombre: String
- edad: int
+ Estudiante(id: String, nombre: String, edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}

class "Resultado" {
- exito: boolean
- mensaje: String
+ esExitoso(): boolean
+ getMensaje(): String
}

FormularioDatosEstudiante --|> ComponenteCrudEstudiante
TablaEstudiantes --|> ComponenteCrudEstudiante
PanelAccionesEstudiante --|> ComponenteCrudEstudiante

MediadorCrudEstudiante ..|> IMediadorCrudEstudiante

ComponenteCrudEstudiante --> IMediadorCrudEstudiante : notifica eventos

MediadorCrudEstudiante o-- FormularioDatosEstudiante
MediadorCrudEstudiante o-- TablaEstudiantes
MediadorCrudEstudiante o-- PanelAccionesEstudiante
MediadorCrudEstudiante --> ControlEstudiante : solicita operación CRUD

ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena

ControlEstudiante --> Resultado
MediadorCrudEstudiante --> Resultado

@enduml

```

